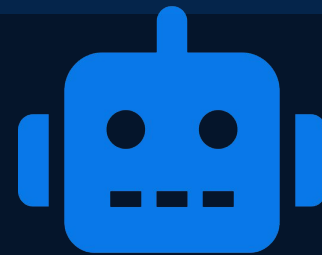


Adversarial Alpha Discovery



An Agentic Multi-Agent Framework for Autonomous Scientific
Discovery in Quantitative Finance



Python

LangGraph

Backtrader

ChromaDB

yfinance

edgartools

Why Traditional Quant Research is Failing

The industry faces four compounding crises that demand an autonomous solution.



Alpha Decay

Profitable inefficiencies disappear once discovered. Competitive markets absorb information faster than humans can generate new edge.



Overfitting

Researchers inadvertently fit strategies to historical noise—models excel in backtests but collapse in live markets.



Look-Ahead Bias

Future information unintentionally leaks into training data, creating unrealistic performance expectations.



Human Bottlenecks

Limited bandwidth, slow hypothesis cycles, and cognitive bias throttle the pace of discovery.

Local Data Lake Architecture

S&P 500 · 10 years · OHLCV · yfinance

yfinance API

Source

Data Ingestion

Download & Validate

Local Data Lake

Parquet / SQLite

Backtrader Engine

Simulation-Ready



Deterministic Testing

Same dataset every experiment — fully reproducible research.



No External Dependencies

No API downtime, latency, or changing data sources.



Version-Controlled Data

Enables rigorous benchmarking and scientific repeatability.



Faster Backtesting

Local I/O eliminates network overhead.

500 tickers · 10 years · ~1.25M daily bars · OHLCV + Adjusted Close

Extracting Semantic Alpha from SEC Filings

edgartools · 10-K filings · Risk Factors · MD&A

1 Filing Ingestion

2 Text Parsing

3 LLM Interpretation

4 Economic Signal Extraction

5 Strategy Hypothesis

Risk Factors

→ Operational threats

→ Regulatory concerns

→ Competitive risks

MD&A Section

→ Executive commentary

→ Future outlook

→ Strategic priorities

EXAMPLE INSIGHT

Filing signal: "Supply-chain disruptions and inventory uncertainty"

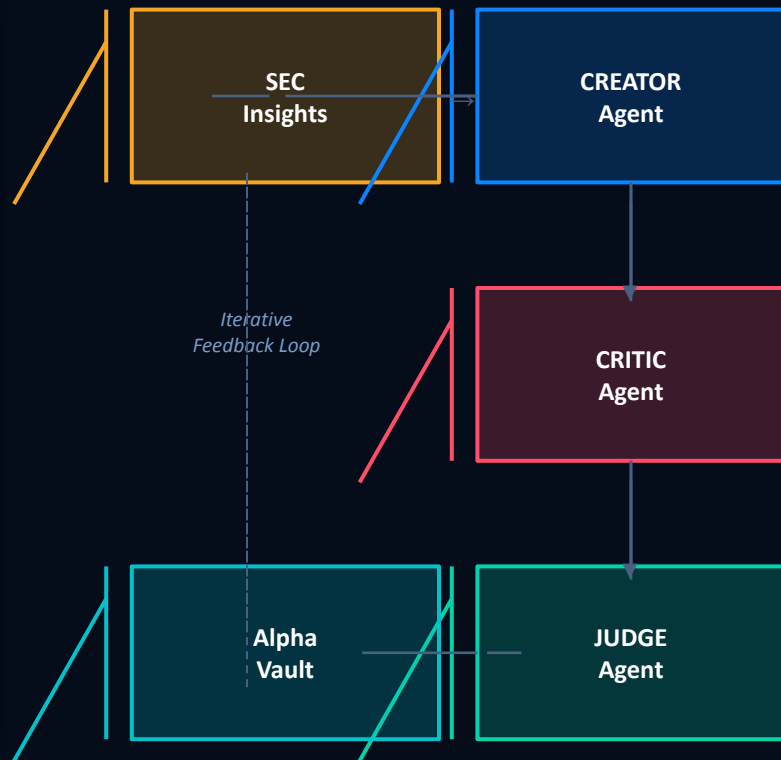
Hypothesis → Firms with elevated supply-chain risk underperform peers during slowdowns.

LangGraph State Machine

Central orchestration engine coordinating all autonomous agents

Research State Object

<code>sec_insights:</code>	SEC Insights
<code>strategy_code:</code>	Strategy Code
<code>backtest_metrics:</code>	Backtest Metrics
<code>critic_analysis:</code>	Critic Analysis
<code>judge_verdict:</code>	Judge Verdict
<code>memory_refs:</code>	Memory References



Secure Strategy Evaluation Environment

Dynamic code execution · Isolated simulation · Standardized metrics

1

Strategy
Generated

2

Code
Compiled

3

Backtrader
Loaded

4

Simulation
Runs

5

Metrics
Returned

Returns**Sharpe****Win Rate****Drawdown**

strategy.py

```
class AlphaStrategy(bt.Strategy):
```

```
    def next(self):
```

```
        # Generated by Creator Agent
```

Isolated Execution

Local env · No internet

Fixed Dataset

Controlled inputs only

Repeatable Tests

Consistent benchmarks

Alpha Generation Engine

Converts data into executable investment strategies



Economic Interpretation

Transforms SEC insights into market narratives and economic hypotheses.



Strategy Synthesis

Generates entry/exit conditions and risk controls from hypotheses.



Code Production

Outputs fully executable Backtrader strategies as Python code strings.

EXAMPLE

INPUT

Rising regulatory risk across multiple filings.



HYPOTHESIS

Regulatory uncertainty → higher volatility → negative future returns.



STRATEGY

Short high-risk firms · Long low-risk peers

The Red Team

A strategy is not robust until it survives adversarial testing.

Transaction Costs

Brokerage commissions on every trade

Slippage

Execution price deviation from signal

Latency

Delayed order execution penalties

Spread Effects

Bid-ask realities on entry & exit

FRAGILITY DETECTION

⚠ Overfitting

⚠ Data Leakage

⚠ Survivorship Bias

⚠ Unrealistic Assumptions

STRESS TEST RESULT

2.10

Original
Sharpe



0.60

After
Friction

✗ STRATEGY REJECTED

Governance & Memory

Logical consistency · Semantic audit · ChromaDB vector memory



Strategy Validity

Economic justification exists for the hypothesis



Logical Consistency

Strategy behavior matches its stated rationale



Experimental Integrity

Proper testing methodology was followed

ChromaDB Vector Memory

Previous Hypotheses

Failed Experiments

Critic Findings

Successful Patterns

MEMORY-DRIVEN DECISION

1. Embed new strategy · 2. Retrieve similar history · 3. Analyze prior failures ·
4. Issue verdict

Phases 1–3: Building the Foundation

PHASE 01

Data Ingestion Layer

- Build local market database
- Download 10 years OHLCV data
- Collect SEC filings with edgartools

✓ Structured Local Data Lake

PHASE 02

Sandbox Engine

- Configure Backtrader engine
- Implement dynamic code execution
- Generate performance reports

✓ Secure Simulation Environment

PHASE 03

Vector Ledger

- Deploy ChromaDB
- Create experiment memory system
- Enable semantic retrieval

✓ Searchable Research History

Phases 4–6: Autonomous Research

PHASE 04

Creator Integration

- Connect LLM reasoning layer
- Automate hypothesis generation
- SEC → strategy pipeline

✓ Working Creator Agent

PHASE 05

Adversarial Loop

- Build Critic Agent
- Introduce market friction testing
- Automate stress evaluation

✓ Robustness Validation Framework

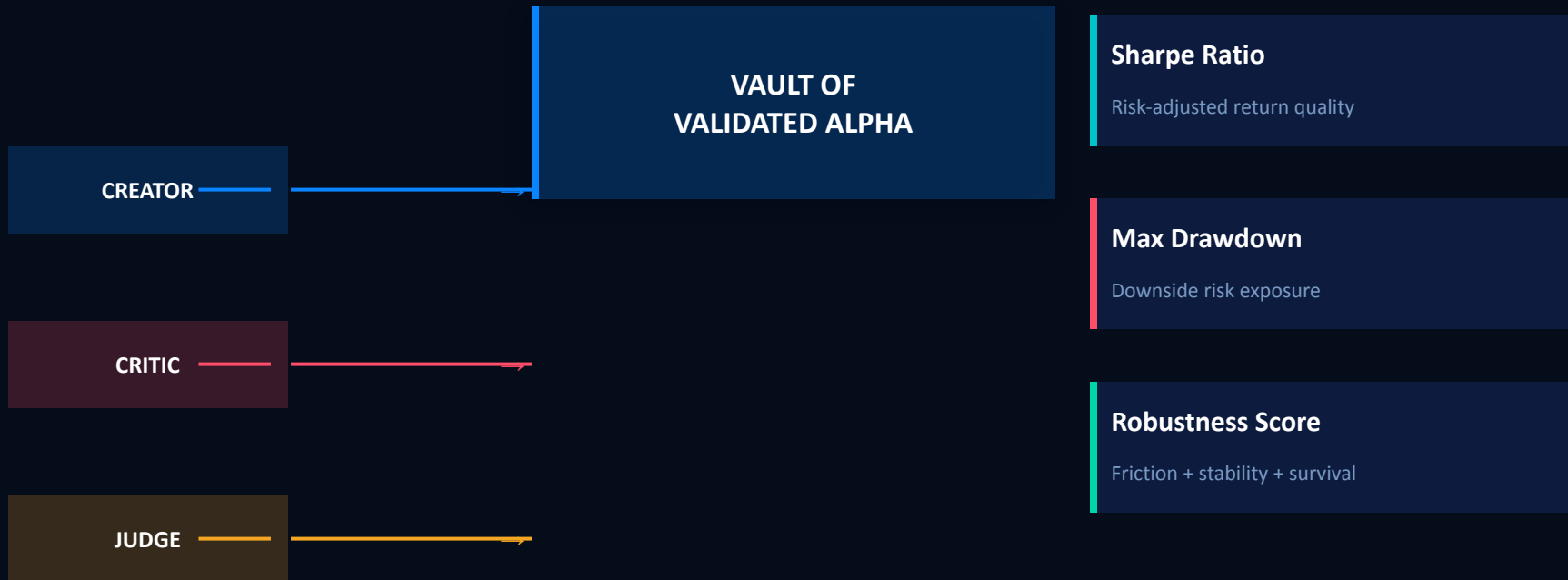
PHASE 06

Orchestration

- Implement LangGraph workflow
- Enable end-to-end autonomy
- Schedule research cycles

✓ Fully Autonomous Quant Lab

From Research Automation to Alpha Manufacturing



An autonomous scientific research system capable of discovering, challenging, and validating investment strategies faster and more rigorously than traditional human-led quantitative research teams.